

1 Lesson 06-10-26

1.1 Exceptions and interrupts

Exceptions are events that modify the normal execution order of the program.

Usually, to refer to events coming from the outside, we use the term **interrupt**. Interrupts come from I/O devices, or reset requests from the device's hardware. Sometimes we see the name *internal interrupt* used for exceptions.

Possible causes of exceptions are:

- I/O device request
- Operating system call by a user program
- Tracing instruction execution
- Breakpoint (programmer-requested interrupt)
- Integer arithmetic overflow or underflow
- FP arithmetic anomaly
- Page fault
- Misaligned memory accesses
- Memory-protection violation
- Undefined instruction
- Hardware malfunction
- Power failure.

Exceptional events (*exceptions, interrupts, or faults*) are more complex to handle in pipelined architectures because several instructions are being executed at a time.

1.1.1 Exception taxonomy

There are a few orthogonal characteristics we can use to classify exceptions.

- **Synchronous vs asynchronous**: an exception is *synchronous* if it can always be triggered in the same position by the code, *asynchronous* if it comes from an external source;
- User **requestable** vs **coerced**: user *requested* exceptions are similar to procedures. *Coerced* exceptions are out of the control of the user program;
- User **maskable** vs **non-maskable**: *maskable* exceptions can be suppressed by the user, whereas *non-maskable* exceptions need to be serviced in any case;
- **Within** vs **between** instructions: exceptions can be raised *within* an instruction (as its executing), or *between* instructions. In this case they are generated by the instruction itself;

- **Resume vs terminate:** after raising an exception, a program can either *resume* execution or *terminate* completely.

So called *restartable machines* are able to handle an exception, save the state, and restart without affecting the execution of the program. Nearly all processors nowadays are restartable machines.

1.1.2 Pipeline

When an exception occurs, the pipeline must execute the following steps:

- Force a trap instruction into the pipeline on the next IF stage;
- until the trap is taken, turn off all writes for the instruction that raised the exception (i.e., the faulty instruction) and for all the following instructions in the pipeline;
- when the exception-handling procedure receives control, it immediately saves the PC of the faulty instruction.

1.1.3 Interrupt protocols

Let's compare *protocols* for handling interrupts on different architectures.

- On **x86**, the general routine is:
 1. An external device sends an interrupt request;
 2. The CPU detects the interrupt;
 3. The CPU reads the Interrupt Type N from the bus;
 4. The CPU saves the value of the processor status word (PSW) and the return address (Code Segment and Instruction Pointer registers) in the stack;
 5. The CPU resets the interrupt flags disabling external interrupts and trap mode;
 6. Using N as an index, the processor reads from the Interrupt Vector Table the address where to jump;
 7. The CPU jumps to the Interrupt Service Routine.
- On **ARM**, the routine changes to:
 1. An external device sends an interrupt request;
 2. The CPU detects the interrupt
 3. The CPU pushes into the current stack the information of the current stack frame composed by 8 registers including the Program Counter and the Processor Status Register;
 4. The CPU updates the processor flags and jumps to the address given by the interrupt type N: on that position a new jump to the actual Interrupt Service Routine may be placed.

For RISC-V, for now let's not focus on interrupt protocol but on possible **causes** of exceptions during each stage of the pipeline.

The fact that all RISC-V instructions write back only in the WB phase (except stores, which write back to memory during the MEM phase) makes pipeline stalling on

Stage	Cause
IF	Pagefault on instruction fetch, Misaligned access, Memory-protection violation
ID	Undefined or illegal opcode
EX	Arithmetic exception
MEM	Page fault on data fetch, Misaligned memory access, Memory-protection violation
WB	None.

exceptions easier. As we saw in 1.2.5, ARM addressing allows for *pre-indexing* and *post-indexing*, which access registers earlier than the writeback phase. This clearly makes the situation more complex.

1.1.4 Precise exceptions

A processor is said to have **precise exceptions** if the pipeline can be stopped so that:

- The instructions just before the instruction that triggered the exception are completed;
- The instructions following the instruction that triggered the exception can be re-started from scratch.

Restarting after an exception may be really hard if exceptions are not handled precisely. Precise exception handling is a must for most architectures, at least for integer instructions. Of course, the problem of precise exception is that they are more difficult to implement / slower.

The main cause for imprecise exceptions is having slower units, such as a *Floating-Point* unit. The solutions in this case can be:

- Accepting imprecise exceptions;
- Providing a fast, but imprecise operating mode, and a slow, but precise one (only one FP instruction is executed at a time);
- Forcing the FP units to early determine whether an instruction can cause an exception, and issuing further instructions only when the previous ones are guaranteed not to cause an exception;
- Buffering the results of each instruction until all the previously issued instructions have been completed.

1.2 Instruction-Level Parallelism

Pipelines exploit the **parallelism** existing among **instructions** (*Instruction-Level Parallelism*, or **ILP**), which allows their execution in parallel. The highest the amount of ILP that can be found and exploited, the better the performance of the pipeline.

There are 2 main ways to exploit ILP:

- **Dynamic**, depending on the hardware to locate parallelism. This dominates both the desktop and embedded market;

- **Static**, depending on the compiler to locate parallelism before the program is even loaded into memory. Can be found in the embedded market, for instance the ARM Cortex-M8. Intel briefly experimented with it in the *Itanium* (IA-64) architecture.

The two approaches can be partly combined.

1.2.1 Basics of ILP

Let's first look at the basics of the *static* kind of ILP.

- The first idea is to scan the code for **basic blocks**. The first kind of ILP is the one among instructions belonging to the same basic block. A basic block is a sequence of instructions with:
 - No branches in, except to the entry
 - No branches out, except at the exit.

Inside a basic block, *rescheduling* can be performed to optimized instruction parallelism. For typical RISC-V programs, the typical size of a basic block is between 4 and 7 instructions. Since these instructions are likely to be dependent one from the other, the amount of parallelism existing within a basic block is normally rather small.

- To further increase the available parallelism, the parallelism among iterations of a **loop** is considered. *Loop unrolling* is the usual technique to avoid branch-related control hazards. Note that unrolling loops by explicitly replicating the loop body multiple times, widens the loop basic block.

In this way:

- The relative overhead due to the control of iteration is reduced;
- The loop body is made wider, thus increasing the chance for the compiler to exploit rescheduling to eliminate stalls.

The disadvantage is that loop unrolling increases the size of the code.

Another possible solution is that of **SIMD** (*Single Instruction, Multiple Data*). More precisely, *Single instruction stream, multiple data streams* may be exploited in:

- **Vector processors**: A vector instruction operates on a set of data, instead of on a scalar data (as a normal instruction);
- *Graphics Processing Units (GPUs)*: Different functional units perform similar tasks in parallel acting on multiple data.

1.3 Dependencies

If two instructions are not dependent, they can be executed in parallel without any stall. If they are dependent, they have to be executed in order (although partly overlapped). Therefore, exploiting the parallelism among instructions requires identifying dependencies existing among them.

There are three kinds of dependencies:

- **Data dependencies:** an instruction j data depends on instruction i if i produces a result that j uses, or there is some transitive chain of instructions which j data depends on and i is a data dependency on;
- **Name dependencies:**
- **Control dependencies:**

At this point we have to make some clarifications. *Dependencies* are properties of the program:

- They create the possibility for a *hazard*;
- Determine the order in which results must be calculated;
- Set an upper bound on the amount of parallelism that can be exploited.

Hazards are potential problems stemming from dependencies in a specific pipeline organization. Stalls depend on the program and the pipeline: a dependency can cause a hazard or not, and the hazard can cause a stall or not (e.g., forwarding can avoid the stall).

1.3.1 Data dependencies

Detecting dependencies involving registers is easy. Detecting dependencies involving memory cells is much more difficult, because accesses to the same cell can look very different. If static techniques are used, the compiler must adopt a conservative approach, assuming that any load instruction refers to the same cell of a previous store. Dependencies involving memory cells can only be detected at run time, when the addresses are known.

1.3.2 Name dependencies

A name dependency occurs when two instructions refer to the same register or memory location (name) but there is no flow of data associated to the name. There are two kinds of name dependencies between an instruction i and an instruction j that follows:

- **Antidependence:** instruction j writes a register or memory location that instruction i reads, and instruction i is executed first;
- **Output dependence:** both instruction i and instruction j write the same register or memory location.

As we know, these dependencies aren't real dependencies and can be eliminated by:

- Increasing the number of registers in the processor;
- Implementing *register renaming*.